



UNIVERSITY OF GHANA

College of Basic and Applied Sciences

School of Engineering Sciences

Department of Computer Engineering

CPEN 208 — Introduction to Software Engineering

Project 2

A REST Web Service Implementing the Database Functions
of the Computer Engineering Department System

Author	Gideon Elorm Glago
Index number	22128981
Programme	BSc Computer Engineering
Level	200
Semester	First Semester, 2025/2026
Service	Express 4 · TypeScript 5 · Node.js
Database	PostgreSQL 16.13
Tests	43 integration tests · all passing

Table of Contents

1. Introduction

- 1.1 What the brief asks for
- 1.2 What "implements the functions" was taken to mean

2. Architecture

- 2.1 Where the logic lives
- 2.2 Request lifecycle
- 2.3 Technology choices

3. API Design

- 3.1 Conventions
- 3.2 The complete endpoint catalogue
- 3.3 Endpoint-to-function mapping

4. The Required Function over HTTP

5. Authentication and Authorisation

- 5.1 JWT issuance
- 5.2 The authorisation matrix

6. Validation and Error Handling

- 6.1 Request validation
- 6.2 Translating SQLSTATE to HTTP

7. Security Measures

8. Testing

- 8.1 Strategy
- 8.2 Results
- 8.3 Tests that matter most

9. Live Demonstration

10. Database Scripts and Backup

11. How to Run It

12. Limitations and Future Work

13. Conclusion

1. Introduction

1.1 What the brief asks for

THE REQUIREMENT, VERBATIM

"Develop an API/web service that implements the functions you created in Project 1." Submission must include the source code of the API/web service, all scripts used to create the database, a backup of the database, and a report.

Project 1 produced a PostgreSQL database, `cpen208_ceds`, holding the Computer Engineering Department's student records, fees, course registrations and teaching assignments — together with 19 callable stored functions, the most important of which returns every student's outstanding fees as a JSON array. Project 2 puts that database on the network.

1.2 What "implements the functions" was taken to mean

There are two readings of the brief. The weak reading is: build an API that *does the same things* the Project 1 functions do — re-deriving the outstanding balance in JavaScript, re-checking class capacity in JavaScript, and so on. The strong reading is: build an API that *calls* those functions, so that the rules exist in exactly one place.

This project takes the strong reading, deliberately. Under the weak reading the system would contain two independent definitions of "outstanding fees" — one in SQL and one in TypeScript — and they would eventually disagree. The bug would surface as the dashboard and the API quoting different balances for the same student, and it would be nobody's obvious responsibility to fix. Under the strong reading that class of bug cannot occur.

THE DESIGN RULE THIS PROJECT FOLLOWS

The API performs **no business arithmetic of its own**. Its responsibilities are transport, authentication, authorisation, input validation and error translation. Every figure it returns and every rule it enforces comes from a Project 1 stored function.

24

ENDPOINTS

16

DB FUNCTIONS
USED

43

TESTS PASSING

5

FUNCTIONALITIES

0

RULES DUPLICATED

2. Architecture

2.1 Where the logic lives

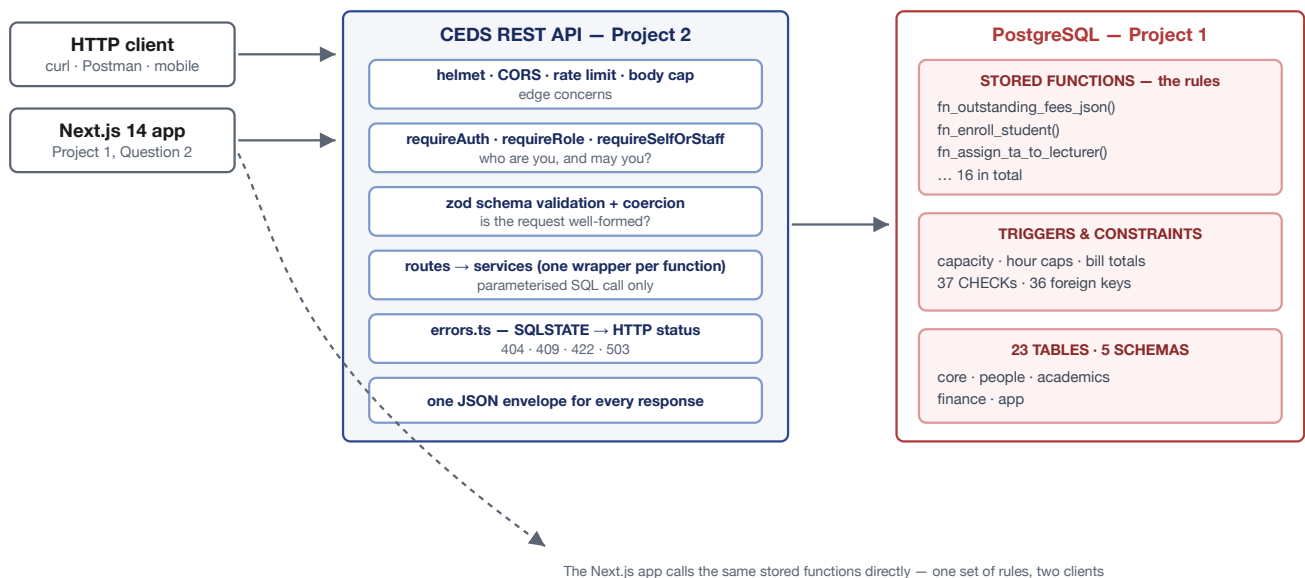


Figure 1 — Layered architecture. The API is a transport and policy layer; every business rule lives in PostgreSQL and is shared with the Next.js application.

2.2 Request lifecycle

A request for the outstanding-fees report passes through seven stages:

1. **Edge middleware** — `helmet` sets security headers, CORS is checked against the allow-list, the rate limiter counts the request, and the body size is capped at 100 kB.
2. **Authentication** — `requireAuth` extracts the Bearer token and verifies its signature, issuer, audience and expiry.
3. **Authorisation** — `requireRole('admin', 'lecturer')` rejects students with 403.
4. **Validation** — the zod schema parses and *coerces* the query string, so `?indebtedOnly=true` becomes a real boolean.
5. **Service** — `getOutstandingFees()` issues one parameterised call: `SELECT finance.fn_outstanding_fees_json($1,$2,$3)`.
6. **Response** — the JSON array is wrapped in the standard envelope with a `count` and the name of the source function.
7. **Errors** — anything thrown along the way reaches one error handler, which maps it to a status code and a machine-readable code.

2.3 Technology choices

Concern	Choice	Why this one
Runtime	Node.js 18+	Same runtime as the Next.js app, so one toolchain across both projects
Framework	Express 4	Small, explicit, and its middleware model maps cleanly onto the seven stages above
Language	TypeScript, strict	The shapes returned by the database functions are declared once and checked at compile time
Database driver	pg with a pool	No ORM: the queries are stored functions, and an ORM would add a translation layer over SQL written by hand on purpose
Auth	jsonwebtoken (HS256)	Stateless tokens suit an API consumed by many clients; the browser app keeps its own revocable sessions
Validation	zod	Strict schemas reject unknown keys instead of ignoring them
Tests	node:test + fetch	No test framework dependency; boots the real app against the real database

3. API Design

3.1 Conventions

The base path is `/api/v1`, versioned from the first commit so a future v2 can run alongside without breaking existing clients. Every successful response shares one envelope, and every failure shares another, so a client never has to guess at the shape:

```
// success
{ "success": true,
  "data": [ ... ],
  "meta": { "count": 36, "source": "finance.fn_outstanding_fees_json()",
            "timestamp": "2026-08-03T20:18:09.793Z" } }

// failure
{ "success": false,
  "error": { "code": "CHECK_VIOLATION",
            "message": "Assignment would put TA 3 at 40 hours/week, above the contracted maximum of
12" },
  "meta": { "path": "/api/v1/teaching/ta-assignments", "method": "POST",
            "timestamp": "2026-08-03T20:18:09.793Z" } }
```

`GET /api/v1` returns a self-describing index of every endpoint at runtime, so the service documents itself rather than relying on documentation that can go stale.

3.2 The complete endpoint catalogue

Method	Path	Access	Purpose
GET	/api/v1	public	Endpoint index
GET	/api/v1/health	public	Liveness + database connectivity
GET	/api/v1/stats	staff	Dashboard counters
POST	/api/v1/auth/register	public	Create an account
POST	/api/v1/auth/login	public	Obtain a JWT
GET	/api/v1/auth/me	any	Resolve the current token
Functionality 1 — Student personal information			
GET	/api/v1/students	staff	Student directory, filterable
GET	/api/v1/students/:studentId	self or staff	Full personal record
Functionality 2 — Student fees payments			
GET	/api/v1/fees/outstanding required	staff	Outstanding fees for every student
GET	/api/v1/fees/summary	staff	Department collection statistics
GET	/api/v1/students/:studentId/fees	self or staff	Full fee statement
GET	/api/v1/students/:studentId/balance	self or staff	Balance only
POST	/api/v1/fees/bills	admin	Generate a bill
POST	/api/v1/fees/payments	admin	Record a payment
Functionality 3 — Course enrollment			
GET	/api/v1/enrollments/offerings	any	Catalogue with seat counts
GET	.../offerings/:offeringId/class-list	staff	Class register
GET	/api/v1/students/:studentId/enrollments	self or staff	A student's courses
POST	/api/v1/enrollments	self or admin	Register for a course
DEL	/api/v1/enrollments	self or admin	Withdraw from a course
Functionality 4 — Lecturer to course assignment			
GET	/api/v1/teaching/lecturers	any	All lecturers
GET	/api/v1/teaching/assignments	any	All allocations
GET	.../lecturers/:lecturerId/workload	any	Workload with nested TAs
POST	/api/v1/teaching/assignments	admin	Assign a lecturer
Functionality 5 — Lecturer to TA assignment			
GET	/api/v1/teaching/assistants	any	TA pool with committed hours
GET	/api/v1/teaching/ta-assignments	any	All TA assignments

Method	Path	Access	Purpose
POST	/api/v1/teaching/ta-assignments	admin / lecturer	Assign a TA
Reference data			
GET	/api/v1/reference/programmes	any	Degree programmes
GET	/api/v1/reference/semesters	any	Academic calendar

3.3 Endpoint-to-function mapping

This is the table that answers the brief most directly: each endpoint and the Project 1 function it invokes.

Endpoint	Project 1 stored function
GET /fees/outstanding	finance.fn_outstanding_fees_json()
GET /fees/summary	finance.fn_fees_summary_json()
GET /students	people.fn_students_json()
GET /students/:id	people.fn_student_profile_json()
GET /students/:id/fees	finance.fn_student_fee_statement_json()
GET /students/:id/balance	finance.fn_student_outstanding_balance()
POST /fees/bills	finance.fn_generate_student_bill()
POST /fees/payments	finance.fn_record_payment()
GET /students/:id/enrollments	academics.fn_student_enrollments_json()
GET .../class-list	academics.fn_class_list_json()
POST /enrollments	academics.fn_enroll_student()
DELETE /enrollments	academics.fn_drop_enrollment()
GET .../lecturers/:id/workload	academics.fn_lecturer_workload_json()
POST /teaching/assignments	academics.fn_assign_lecturer_to_course()
GET /teaching/ta-assignments	academics.fn_ta_assignments_json()
POST /teaching/ta-assignments	academics.fn_assign_ta_to_lecturer()
POST /auth/register	app.fn_register_user()
GET /auth/me	app.fn_user_context_json()
GET /stats	app.fn_dashboard_stats_json()

The remaining endpoints — the course catalogue, the lecturer list, the TA list and the reference data — read the Project 1 *views* (`v_course_offering_summary` , `v_lecturer_course_allocation`) rather than functions, for the same reason: the join is already written and tested in the database.

4. The Required Function over HTTP

The headline endpoint returns, unmodified, the JSON array produced by the Project 1 function. The entire service-layer implementation is four lines:

```
export function getOutstandingFees(filters: {
  academicYearId?: number; programmeId?: number; indebtedOnly?: boolean;
}) {
  return callJson('finance.fn_outstanding_fees_json($1, $2, $3)', [
    filters.academicYearId ?? null,
    filters.programmeId ?? null,
    filters.indebtedOnly ?? false,
  ]);
}
```

There is no `reduce`, no subtraction and no rounding in TypeScript. The three optional query parameters map one-to-one onto the function's three optional arguments, so the HTTP surface offers exactly the flexibility the function already had, and nothing more.

Request

```
GET /api/v1/fees/outstanding?indebtedOnly=true HTTP/1.1
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Response (abridged)

```
{
  "success": true,
  "data": [
    {
      "student_id": 3,
      "student_number": "21047733",
      "full_name": "Belinda Nana Yaa Addo",
      "email": "belinda.addo@st.ug.edu.gh",
      "programme": "BSc Computer Engineering",
      "level": 400,
      "currency": "GHS",
      "total_billed": 6880,
      "total_paid": 0,
      "outstanding_balance": 6880,
      "percentage_paid": 0,
      "payment_status": "NO PAYMENT",
      "last_payment_date": null,
      "bill_count": 1,
      "bills": [
        { "bill_id": 3, "bill_reference": "BILL-20252026-00003",
          "academic_year": "2025/2026", "amount_billed": 6880,
          "amount_paid": 0, "balance": 6880, "due_date": "2025-10-31",
          "bill_status": "overdue", "payments_recorded": 0, "is_overdue": true }
      ]
    }
    // ... 35 more students
  ],
  "meta": { "count": 36,
    "source": "finance.fn_outstanding_fees_json()",
    "filters": { "indebtedOnly": false },
    "timestamp": "2026-08-03T20:17:37.9Z" }
}
```

The `meta.source` field names the database function that produced the payload. That is not decoration: it makes the claim in §1.2 checkable by anyone reading a response, and it means a consumer investigating a suspicious figure knows exactly which SQL to read.

5. Authentication and Authorisation

5.1 JWT issuance

`POST /api/v1/auth/login` accepts a username, index number or e-mail with a password. The service loads the account, compares the supplied password against the stored bcrypt hash, and on success signs a JWT containing the user id, username, role, and — importantly — the resolved `studentId` and `lecturerId`.

Carrying those two ids in the token is what makes per-record authorisation cheap: deciding whether a caller may read `/students/42` is then a comparison against the token, not an extra database round trip on every request.

An unknown username and a wrong password produce the *identical* response. The unknown-user branch even performs a dummy bcrypt comparison, so the two paths take comparable time and the endpoint cannot be used to discover which accounts exist. This is asserted by a test that compares the two error messages directly.

5.2 The authorisation matrix

Capability	Student	Lecturer	TA	Admin
Read own profile, fees, courses	yes	yes	yes	yes
Read another student's record	no	yes	no	yes
List the student directory	no	yes	no	yes
Outstanding-fees report	no	yes	no	yes
Register / withdraw self	yes	—	no	yes
Register another student	no	no	no	yes
Read a class list	no	yes	yes	yes
Assign a lecturer to a course	no	no	no	yes
Assign a TA	no	own courses only	no	yes
Record a payment / generate a bill	no	no	no	yes

Three guards implement the whole matrix. `requireAuth` establishes identity; `requireRole(...)` handles the coarse cases; and `requireSelfOrStaff('studentId')` handles the row-level case — a student may read `/students/:id` only when the `id` in the path equals the `studentId` in their own token.

6. Validation and Error Handling

6.1 Request validation

Every body, query string and path parameter is parsed by a zod schema before a handler runs. The schemas are **strict**: an unrecognised key is rejected rather than silently ignored, so a client that misspells a field learns about it immediately instead of wondering why the value had no effect. Validation also *coerces*, which matters because everything in a URL is a string — `?indebtedOnly=true` reaches the handler as a boolean and `/students/42` as a number.

```
POST /api/v1/fees/payments
{ "studentId": 1, "billId": 1, "amount": -100, "method": "bitcoin" }

→ HTTP 400
{ "success": false,
  "error": {
    "code": "BAD_REQUEST",
    "message": "Invalid request body.",
    "details": [
      { "field": "amount", "message": "Too small: expected number to be >0" },
      { "field": "method",
        "message": "Invalid option: expected one of \"bank_transfer\"|\"mobile_money\"|\"...\" }
      ] } }
```

Note that *both* problems are reported at once. Returning only the first error would force a client into a guess-and-retry loop.

6.2 Translating SQLSTATE to HTTP

Because the rules live in the database, the API must turn a database rejection into a sensible HTTP answer. The Project 1 functions raise errors with meaningful SQLSTATE codes, and `src/errors.ts` maps them:

SQLSTATE	Meaning	HTTP	Example
P0002	no_data_found	404	Unknown student id
23505	unique_violation	409	Duplicate registration; second lead lecturer
23503	foreign key violation	409	Reference to a non-existent offering
23514	check_violation	422	TA over their contracted weekly hours
22P02	invalid input syntax	400	A non-numeric id reaching the database
08006 , 57P01	connection failure	503	Database restarting

Anything unmapped becomes a 500 with a generic message — the raw driver error is logged server-side but never returned, so internal structure is not leaked to a caller. Stack traces appear in responses only when `NODE_ENV` is not `production`.

THE PAYOFF

"A teaching assistant may not exceed their contracted weekly hours" is written once, in `academics.fn_assign_ta_to_lecturer`. It is enforced identically for `psql`, for the Next.js dashboard, and for an HTTP client — and through HTTP it arrives as a correct **422 Unprocessable Entity** carrying the database's own explanatory message. Nothing about that rule is written in TypeScript.

7. Security Measures

Control	Implementation
SQL injection	Every value is a bound parameter (<code>\$1</code> , <code>\$2</code> , ...). No user input is ever concatenated into SQL.
Password storage	<code>bcrypt</code> , cost 10. Plain passwords are never stored, logged, or returned.
Account enumeration	Unknown user and wrong password return identical responses; a dummy hash comparison equalises timing.
Brute force	Global limit 300 requests / 15 min; 20 on <code>/auth/*</code> . Failed attempts are counted on the account.
Transport headers	<code>helmet</code> with a strict Content-Security-Policy; <code>x-powered-by</code> disabled.
CORS	Explicit allow-list from <code>CORS_ORIGINS</code> ; the wildcard is a development default only.
Payload size	Bodies capped at 100 kB — nothing this API accepts is large.
Token integrity	Issuer and audience are verified alongside the signature, so a token minted for another service is rejected.
Secret management	<code>JWT_SECRET</code> is <i>mandatory</i> in production — the service refuses to start without it. <code>.env</code> is git-ignored.
Least privilege	Guards on every non-public route; students are confined to their own rows.
Auditability	Logins, registrations, enrolments, payments and assignments are written to <code>app.audit_log</code> .
Information leakage	Unmapped database errors return a generic message; stack traces are suppressed in production.

8. Testing

8.1 Strategy

The suite contains **integration** tests, not mocked unit tests. That is a deliberate choice: the entire claim of this project is that the API faithfully exposes the Project 1 database functions. A mock of the database would be a mock of the very thing under test, and would pass happily while the real SQL was broken. The tests therefore boot the real Express application against the real `cpen208_ceds` database.

The app is exported from `src/app.ts` separately from `src/index.ts` , which owns `listen()` . This lets the suite start the service on an ephemeral port in-process rather than spawning a second one. The suite is read-mostly; the single write it performs — registering a throwaway account — is removed in the teardown hook.

8.2 Results

```
$ npm test

▶ service metadata          ✓ 3 tests
▶ authentication            ✓ 7 tests
▶ functionality 1 – student personal information ✓ 6 tests
▶ functionality 2 – student fees payments      ✓ 9 tests
▶ functionality 3 – course enrollment          ✓ 6 tests
▶ functionality 4 – lecturer to course assignment ✓ 5 tests
▶ functionality 5 – lecturer to TA assignment  ✓ 4 tests
▶ reference data                ✓ 1 test

i tests 43
i pass 43
i fail 0
i duration_ms 1231.32
```

43

TESTS

43

PASSING

0

FAILING

8

SUITES

1.2 s

RUNTIME

8.3 Tests that matter most

Any suite can assert that a 200 is a 200. These are the cases that would actually catch a regression:

Test	What it protects against
Re-derives <code>outstanding = billed – paid</code> for every row of the report	A silent arithmetic regression in the SQL — the single most consequential figure in the system
Asserts no student's balance is ever negative	The exact double-payment bug found during Project 1 development
Asserts the fee statement and the balance endpoint agree	Two functions drifting apart
Confirms pending/reversed payments are excluded, by querying the fixture first	The central fees rule; the test fails loudly if the fixture ever stops containing an unconfirmed payment
<code>seats_left = capacity – enrolled</code> for every offering	An inconsistent view definition
Duplicate registration returns 409	The unique constraint no longer reaching the client correctly
A second lead lecturer returns 409	The partial unique index being dropped
Over-committing a TA returns 422	The hour cap in the stored function being bypassed
A student reading another student's record returns 403	A broken row-level guard — a genuine data-protection failure
Unknown user and wrong password return identical messages	Account enumeration creeping back in
An unknown key in the body returns 400	Schemas quietly becoming permissive

Several of these assert on the *database's* behaviour reaching the client intact. That is the point: they would fail if someone "helpfully" reimplemented a rule in TypeScript and let the two definitions diverge.

The transcript below was captured from the running service and is reproduced in full in `docs/api_demo_output.txt`. It shows a successful login, the required deliverable over HTTP, and two rejections.

[illegible]

```
$ curl -s http://localhost:4000/api/v1/fees/outstanding -H "Authorization: Bearer $TOKEN"
```

```
Array?           : true
Students in array: 36
Source function   : finance.fn_outstanding_fees_json()
```

```
$ curl -s http://localhost:4000/api/v1/fees/outstanding -H "Authorization: Bearer $STUDENT_TOKEN"

HTTP 403
{ "success": false,
  "error": { "code": "FORBIDDEN",
             "message": "This endpoint requires one of these roles: admin, lecturer. Your role is student." },
  "meta": { "path": "/api/v1/fees/outstanding", "method": "GET", "timestamp": "..." } }
```

```
$ curl -s -X POST http://localhost:4000/api/v1/teaching/ta-assignments \
-H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
-d '{"lecturerId":4,"taId":3,"semesterId":3,"weeklyHours":40}'

HTTP 422
{ "success": false,
  "error": { "code": "CHECK_VIOLATION",
             "message": "Assignment would put TA 3 at 40 hours/week, above the contracted maximum of 12" },
  "meta": { "path": "/api/v1/teaching/ta-assignments", "method": "POST", "timestamp": "..." } }
```

That last response is the clearest evidence of the architecture working as intended. Nothing in the TypeScript knows that TA/2025/003 is contracted for twelve hours a week. The API forwarded the request, PostgreSQL

refused it with `check_violation`, and the error handler turned that into a **422** carrying the database's own message.

10. Database Scripts and Backup

The brief requires this submission to include the scripts used to create the database and a backup of it. Both are in `project-2/database/`, a copy of the Project 1 scripts so that Project 2 stands alone. It is the *same* database — built once, used by both.

Path	Contents
<code>database/00...07_*.sql</code>	Eight ordered scripts: database, schemas, tables, indexes and triggers, functions, views, seed data, verification
<code>database/run_all.sh</code>	Builds the whole database in one command
<code>database/backup/cpen208_ceds_backup.sql</code>	Plain SQL, 244 KB — <code>psql -d newdb -f ...</code>
<code>database/backup/cpen208_ceds_backup.dump</code>	Custom format, 200 KB — <code>pg_restore -d newdb --no-owner ...</code>
<code>database/backup/cpen208_ceds_schema_only.sql</code>	Structure without data
<code>database/backup/cpen208_ceds_data_only.sql</code>	Data without structure

The plain-SQL backup was restored into a scratch database and the required function re-run there, returning all 36 students. A backup that has never been restored is only a hope.

11. How to Run It

```
# 1 – build the database (skip if Project 1 already did this)
cd project-2/database && ./run_all.sh

# 2 – configure and start the service
cd ../api
cp .env.example .env          # set DATABASE_URL and JWT_SECRET
npm install
npm run dev                   # http://localhost:4000

# 3 – run the tests
npm test                       # 43 integration tests

# 4 – explore
curl http://localhost:4000/api/v1      # self-describing index
curl http://localhost:4000/api/v1/health # liveness
```

The service performs a database pre-flight check at startup and exits with a clear message if PostgreSQL is unreachable. A service that starts "successfully" and then returns 500 on every request is worse than one that refuses to start.

12. Limitations and Future Work

Limitations

- Tokens are stateless and therefore not individually revocable before expiry; the two-hour lifetime bounds the exposure. The browser application uses revocable server-side sessions instead.
- Rate limiting is in-process, so it resets on restart and is not shared across instances. A multi-instance deployment would need a Redis-backed store.
- The endpoint index is hand-maintained rather than generated; a machine-readable OpenAPI document would be better.
- List endpoints return complete result sets. At the current scale (36 students) that is correct; at thousands it would need cursor pagination.
- There is no refresh-token flow, so a client must re-authenticate every two hours.

Future work

- Generate an OpenAPI 3.1 document from the existing zod schemas, and serve Swagger UI from it — the schemas already contain everything required.
- Add refresh tokens with rotation and reuse detection.
- Move rate limiting to a shared store so the service can scale horizontally.
- Add structured request logging with correlation ids to tie an HTTP request to its `app.audit_log` rows.
- Enable PostgreSQL row-level security so the database enforces the "students see only their own rows" rule even if an API guard were ever removed.

13. Conclusion

The brief for Project 2 asked for an API that implements the functions created in Project 1. That has been delivered as a versioned REST service exposing 24 endpoints across all five required functionalities, with JWT authentication, a role- and row-level authorisation model, strict input validation, a coherent error contract, and 43 passing integration tests.

The defining decision was to treat "implements" as *calls* rather than *reimplements*. Every figure the service returns and every rule it enforces comes from a PostgreSQL stored function written in Project 1. The clearest demonstration is the 422 in §9: rejecting an over-committed teaching assistant required no TypeScript at all, because the rule was already in the database and the API's only job was to carry the refusal back to the caller with the right status code.

The result is that the Next.js application from Project 1 and this REST service are two faces of one system rather than two systems that happen to share a database. Neither can quote a different outstanding balance from the other, because neither computes one.